

Circular Dependency & the @Required Annotation

Spring Framework Notes · Day 19 · Annotations (Part 2)

In this session, we look at a classic Spring interview problem: what happens when three beans depend on each other in a circular chain. We solve it step by step, moving from constructor injection to setter injection, and along the way we discover a hidden gotcha with the **@Required** annotation and how **BeanPostProcessor** quietly does the work behind every annotation we use.

STEP 1 | Constructor Injection - The Problem Statement

Consider three classes: **Class A** depends on **Class B**, **Class B** depends on **Class C**, and **Class C** depends back on **Class A**. This forms a circular chain: $A \rightarrow B \rightarrow C \rightarrow A$.

Interview Question

Suppose Class A has an object of Class B, Class B has an object of Class C, and Class C has an object of Class A. What problem can occur if we use constructor injection for all three?

What happens internally

```
Spring starts creating bean A | A needs B -  
--> Spring starts creating bean B | B needs  
C ---> Spring starts creating bean C | C  
needs A ---> But A is still being created!
```

Since A is not fully built yet, Spring cannot hand a reference to it. This gives us the output below.

OUTPUT

```
BeanCurrentlyInCreationException
```

STEP 2 | Reproducing the Problem with Constructor Injection

Java classes

A.java

```
public class A {  
    B b;  
  
    public A(B b) {  
        System.out.println("A created");  
    }  
}
```

B.java

```
public class B {  
    C c;  
  
    public B(C c) {  
        System.out.println("B created");  
    }  
}
```

C.java

```
public class C {  
    A a;  
  
    public C(A a) {  
        System.out.println("C created");  
    }  
}
```

application-context.xml

```
<bean id="a" class="com.demo.A">    <constructor-arg ref="b"/>  
</bean>  
  
<bean id="b" class="com.demo.B">    <constructor-arg ref="c"/>  
</bean>  
  
<bean id="c" class="com.demo.C">    <constructor-arg ref="a"/>  
</bean>
```

OUTPUT

```
BeanCurrentlyInCreationException
```

STEP 3 | The Fix - Switch to Setter Injection

Constructor injection creates the object and injects its dependency in the same step, so all three beans need each other to already exist at creation time. That is exactly why the circle breaks.

Setter injection separates the two steps: Spring first creates the object with an empty constructor, and only afterwards calls the setter to inject the dependency. This means all three beans can be created first, and wired together second.

Revised flow

```
Step 1 - create the objects Create A [ok]  
        Create B [ok] Create C [ok]  
  
Step 2 - wire the dependencies  
        a.setB(b)  
        b.setC(c)  
        c.setA(a)
```

STEP 4 | The New Problem with Setter Injection

Setter injection solves the circular dependency, but it introduces a different weakness:

- Constructor injection makes a dependency **mandatory**.
- Setter injection makes a dependency **optional** by default.

Student.java

```
public class Student {  
  
    private Course course;  
  
    public void setCourse(Course course) {  
        this.course = course;    }  
  
    public void show() {  
        System.out.println(course.getName());  
    }  
}
```

Course.java

```
public class Course {  
  
    private String name = "Spring Framework by Prashant";  
  
    public String getName() {  
        return name;    } }  

```

application-context.xml

```
<bean id="student" class="com.demo.Student"/>  
  
<bean id="course" class="com.demo.Course"/>  
  
<!-- Notice: no property tag connects student to course -->
```

Even though the course property is never injected, Spring still creates the student bean without complaint.

OUTPUT

```
Student object created successfully. (course == null)
```

WHY: Spring allows this because, by default, setter-injected dependencies are treated as optional. If we want to force a setter dependency to be mandatory, we need a way to tell Spring so.

STEP 5 | Introducing @Required

The **@Required** annotation tells Spring: "this setter method must be called - do not treat it as optional." **Student.java**

```
import org.springframework.beans.factory.annotation.Required;

public class Student {

    private Course course;

    @Required
    public void setCourse(Course course) {
        this.course = course;    } }
}
```

To try this out, the Spring Core dependency version in pom.xml was changed as shown below.

Before

pom.xml

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-core</artifactId>
    <version>6.2.18</version>
<scope>compile</scope>
</dependency>

<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>6.2.18</version>
<scope>compile</scope>
</dependency>
```

After

pom.xml

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-core</artifactId>
    <version>5.0.20.RELEASE</version>
    <scope>compile</scope>
</dependency>

<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>5.0.20.RELEASE</version>
    <scope>compile</scope>
</dependency>
```

Running the program again gave a surprising result:

OUTPUT

No exception!

Question: Why did @Required not work here?

STEP 6 | BeanPostProcessor - The Missing Piece

This is the key idea behind the answer: **annotations do nothing on their own**. Simply placing `@Required` above a method does not make Spring enforce anything. Spring needs a helper object, called a **BeanPostProcessor**, to actually read the annotation and act on it.

A simple way to picture it

```
Bean ---> Inspector (BeanPostProcessor) ---> Ready Bean
```

Or, in more detail:

```
Bean
| v
BeanPostProcessor checks the
annotations | v
Spring Container
```

How a BeanPostProcessor works

Every time Spring creates a bean, it passes that bean to all registered BeanPostProcessors. Each processor inspects the bean and checks whether certain annotations are present, such as:

- `@Required`
- `@Autowired`
- `@PostConstruct`
- `@PreDestroy`

If a relevant annotation is found, the processor performs the extra work associated with it. For **@Required** specifically, it checks one simple thing: was the setter method actually called?

- **Yes** → bean creation continues normally.
- **No** → Spring throws an exception.

Full internal flow

```
Spring starts |
Creates the Student bean | Sends the
bean to BeanPostProcessor |
BeanPostProcessor scans for annotations
|
@Required is found | Checks whether the
dependency was injected |
Bean is accepted, or an exception is
thrown
```

ANSWER: This explains Step 5: without a BeanPostProcessor registered for `@Required`, no one is checking the annotation - so Spring happily creates the bean even with a missing dependency.

STEP 7 | Enabling @Required

To make Spring actually enforce `@Required`, we must register the matching `BeanPostProcessor` as a bean in the configuration file.

application-context.xml

```
<bean class=
    "org.springframework.beans.factory.annotation.RequiredAnnotationBeanPostProcessor"/>
```

With this processor registered, Spring now checks both whether `@Required` is present and whether the corresponding setter was called.

OUTPUT

```
BeanInitializationException: Property 'course' is required.
```

STEP 8 | The Recommended Approach

Registering a separate `BeanPostProcessor` bean for every annotation quickly becomes tedious. Spring provides a single configuration tag that automatically enables all the common annotation-processing `BeanPostProcessors` together:

- `@Autowired`
- `@Required`
- `@PostConstruct`
- `@PreDestroy`
- `@Resource`

Instead of registering `BeanPostProcessors` one by one, simply add:

```
<context:annotation-config/>
```

This tag requires the context namespace to be declared in the XML configuration file, as shown below.

application-context.xml

```
<beans
    xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"

    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        https://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
        https://www.springframework.org/schema/context/spring-context.xsd">
```

IMPORTANT NOTE FOR STUDENTS

@Required is **deprecated** in modern versions of Spring. It still matters for us because it is an excellent way to understand three core ideas:

- Spring Core internals
- The bean life cycle
- How a BeanPostProcessor works